Raportti: Rosa Huotari

Osa 1: GitHub Copilotin käyttöönotto ja agentin määrittely

En ollut aiemmin käyttänyt GitHub Copilotia. Käytin tehtävänannossa annettuja linkkejä ohjeistuksiin ja lisäksi Moodlessa ollut video “Using git with VS Code” auttoi pääsemään alkuun projektissa. Sain vahvistettua opiskelijastatuksen alkuvaikeuksien jälkeen. Ohjeistukset olivat selkeät, mutta GitHubin lomake vahvistukselle oli hankala, sillä en löytänyt mahdollisuutta liittää esim. näyttökuvaa tai tiedostoa hakemukseen. Kysyin opetusavustajalta neuvoa ja hän ohjeisti ottamaan kuvan esim. laitoksen kulkukortista, sillä muut dokumentit, joita olin yrittänyt eivät olleet saaneet hakemusta menemään läpi. Kulkukortin kuvalla sain hakemuksen tehtyä ja opiskelijastatuksen vahvistuksen hyväksytysti läpi. Oma osaamispohjani kurssin aiheesta ja ylipäätään koodaamisesta on hyvin hatara, joten muokkasin agentin olettamaan, että opiskelija ei tiedä aiheesta paljoakaan. Tahdoin agentin kysyvän yksityiskohtaisia kysymyksiä, jotta se ei oleta minun tietävän aiheesta asioita, joita en välttämättä ole vielä sisäistänyt. Halusin agentin pitävän työnteon hauskana, käyttämään huumoria ja juttelemaan minulle kuin ystävälle, jotta tehtävän tekeminen olisi mielekästä.

Osa 2: Kehitysprosessi

Työskentely Copilotin kanssa opetti minulle paljon perusasioita web-ohjelmoinnista ja pelin kehittämisestä. Itse pelin rakentaminen ja eri vaiheet olivat mielestäni hyvin selkeät ja Copilot opasti hyvin kunkin vaiheen lävitse. Suurin anti Copilotilta oli kuitenkin sen perusteelliset selitykset kustakin aiheesta niin, että tiesin ja ymmärsin jokaisen koodirivin. Alla kolme kohtaa, joissa sain jonkinlaisen oivalluksen Copilotin kanssa.

1. Pelialueen ruudukon ymmärtäminen: Tämä oli ensimmäinen kerta, kun rakennan itse peliä ja pelialueen ruudukon hahmottaminen oli aluksi haastavaa. Minulla menivät sekaisin ruudun koko sekä ruutujen määrä pelialueella. Copilotin selityksen ja tarkentavien kysymysten avulla oivalsin logiikan. Kuvakaappaukset seuraavalla sivulla.

Mitä tapahtuisi, jos canvas olisi 400 pikseliä leveä mutta `gridSize` olisi 30?

Restore Checkpoint

index.html:27

silloin yhden ruudun koko olisi 13,333..?

3 days

Clarifying misconceptions >

Melkein, mutta tässä menee kaksi asiaa ristiin. 😉

Jos:

```
canvas.width = 400;
gridSize = 30;
```

niin:

```
tileCount = canvas.width / gridSize;
tileCount = 400 / 30;
tileCount = 13.333...
```

Tämä ei tarkoita, että ruudun koko olisi `13,333` pikseliä. Ruudun koko on edelleen:

```
gridSize === 30
```

`tileCount` kertoo, kuinka monta 30 pikselin ruutua mahtuisi 400 pikselin levyiselle alueelle. Koska 30 ei mene tasan 400:aan, alueelle mahtuu:

- 13 kokonaista ruutua eli 390 pikseliä
- jäljelle jää 10 pikseliä

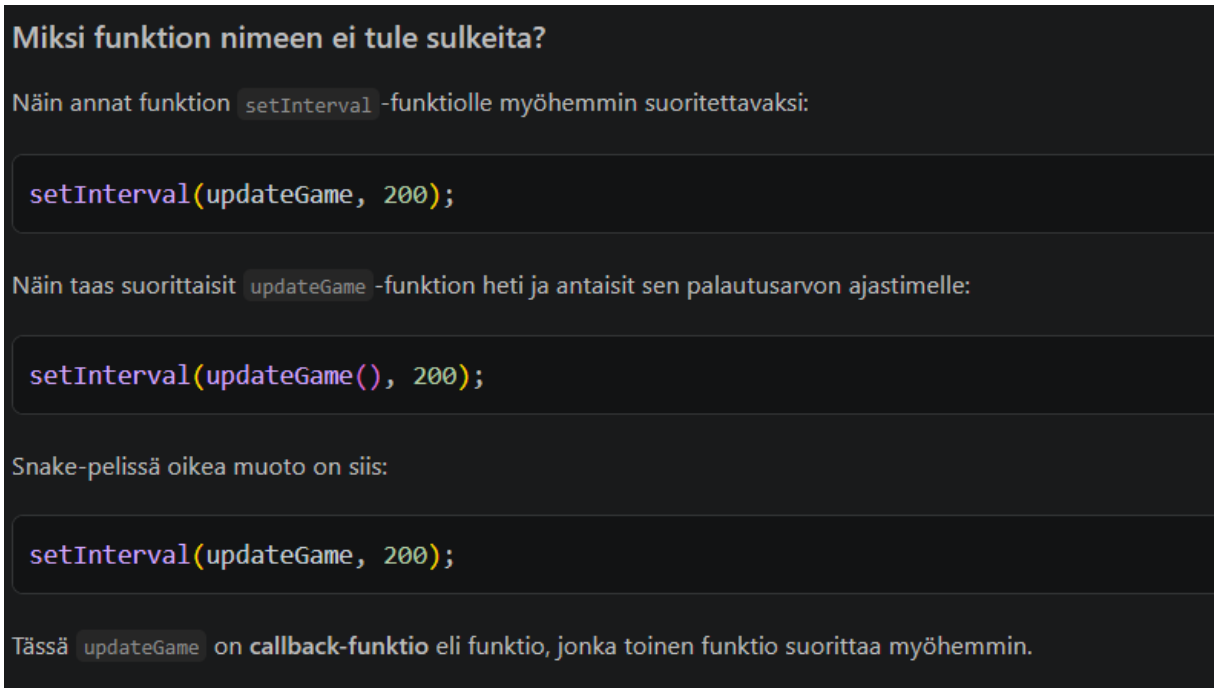
Snake-pelissä koordinaattien pitäisi yleensä olla kokonaislukuja. Siksi `tileCount` kannattaa tällaisessa tilanteessa pyöristää alaspäin:

```
const tileCount = Math.floor(canvas.width
```

Tällöin:

```
Math.floor(400 / 30) === 13
```

2. Callback-funktio sekä sulkujen merkitys ja niiden käyttö: Pyysin Copilotia selittämään setInterval-funktiota tarkemmin ja se selvensi minulle paljon myös callback-funktion käsitettä sekä sulkujen käyttöä. Noudatin ohjetta, sillä muuten pelini ei olisi toiminut toivotulla tavalla.



Miksi funktion nimeen ei tule sulkeita?

Näin annat funktion `setInterval` -funktiolle myöhemmin suoritettavaksi:

```
setInterval(updateGame, 200);
```

Näin taas suorittaisit `updateGame` -funktion heti ja antaisit sen palautusarvon ajastimelle:

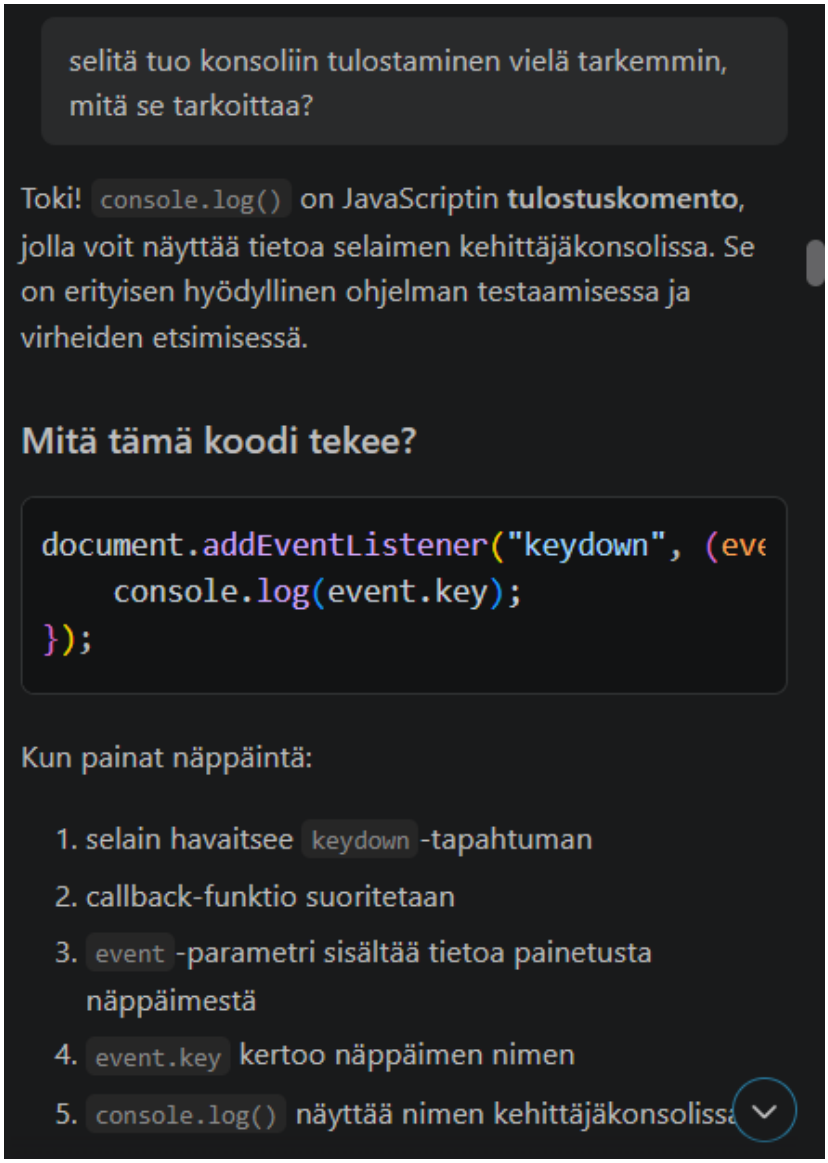
```
setInterval(updateGame(), 200);
```

Snake-pelissä oikea muoto on siis:

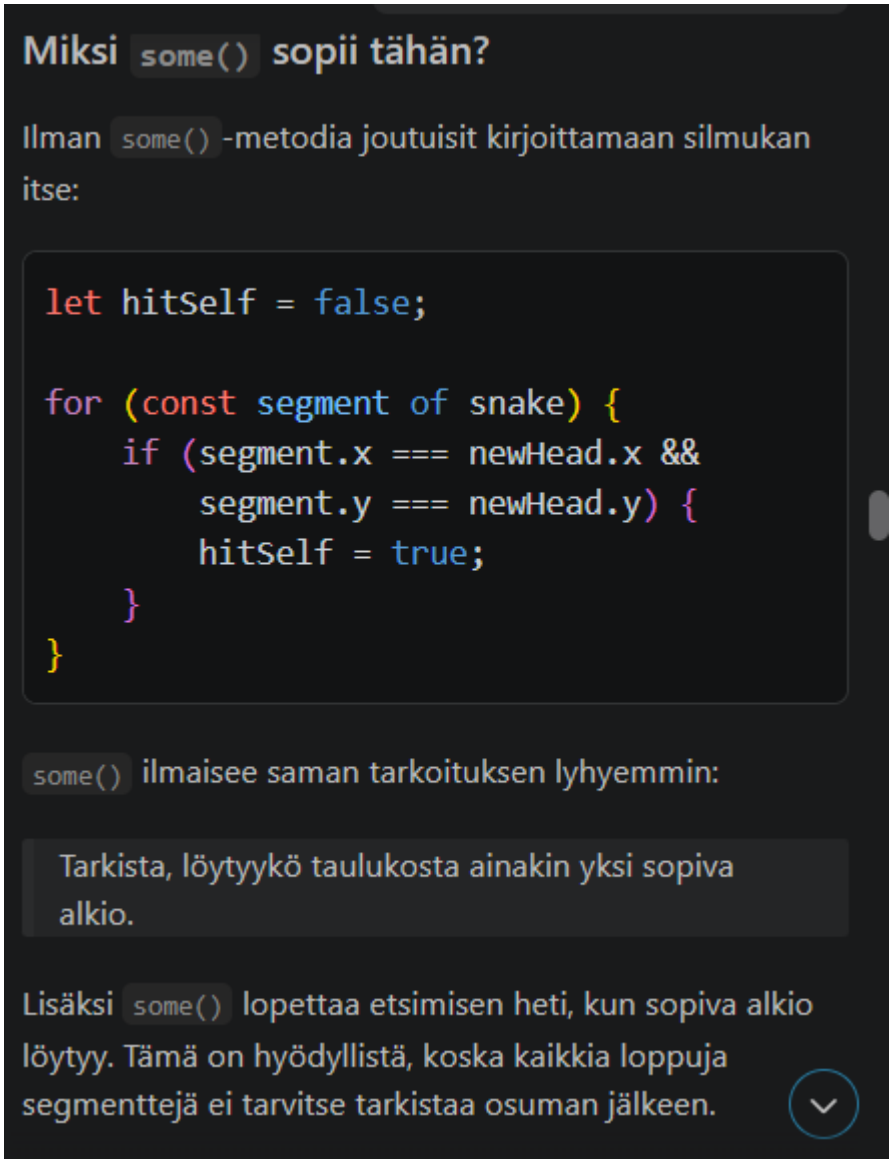
```
setInterval(updateGame, 200);
```

Tässä `updateGame` on **callback-funktio** eli funktio, jonka toinen funktio suorittaa myöhemmin.

3. Kehittäjäkonsoli ja sen tarkastelu: Opin kuinka kehittäjäkonsoli toimii selaimessa ja kokeilin sitä useaan kertaan, jolloin pystyin tarkistamaan pelin toimivuutta jokaisessa vaiheessa. En ollut ennen käyttänyt tätä konsolia. Kuva seuraavalla sivulla.



4. Some-metodi: Vaiheessa 4 määriteltiin, että peli päättyy, jos käärme osuu itseensä. Tämän Copilot ehdotti tehtävän some-metodilla. Mielestäni se kuulosti järkevältä tavalta tarkistaa, osuuko käärmeen pää johonkin sen vartalon osaan. Pyysin Copilotia selittämään metodin tarkemmin ja miksi se sopii tähän tehtävän vaiheeseen. Noudatin Copilotin ohjetta. Kuva seuraavalla sivulla.



Osa 3: Reflektio

Määrittelin agenttini olettamaan, että keskustelukumppanin tietämys aiheesta on hyvin vähäistä jopa olematonta. Agenttini selitti asiat hyvin yksityiskohtaisesti ja helppotajuisesti, mikä oli oman oppimiseni kannalta olennaista ja positiivinen asia. Copilot kuitenkin antoi myös mielestäni aika paljon valmista koodia, joka nopeutti ohjelman kirjoittamista. Ehkä olisin toivonut, että agentti olisi kysellyt enemmän kysymyksiä ja antanut valmiita vastauksia vähemmän. Kun agentti kysyi kysymyksiä, ne auttoivat hyvin omaa ajattelua eteenpäin. Toisaalta, jotta pääsin yhtään tehtävässä eteenpäin, olivat myös valmiit ajatusmallit hyvin tärkeässä roolissa. Olisin ehkä voinut itse kysyä Copilotilta enemmän erilaisia vaihtoehtoisia tapoja ratkaista vaiheet, enkä luottaa vain ensimmäiseen ehdotukseen.

Käytin projektissa mm. luennoilta tuttuja DOM-käsitteitä getElementById ja addEventListener tehdessäni restart-nappia, jolla voidaan aloittaa uusi peli. Lisäksi käytin esimerkiksi funktioita setInterval ja clearInterval, joita en ollut aiemmin käyttänyt ja tämän projektin myötä opin niiden toiminnan. Tekoäly selitti funktiot pala palalta hyvin ymmärrettävästi. TextContent oli myös yksi DOM-käsite, jota en ollut itse aiemmin käyttänyt. Sitä käytin näyttääkseni sivulla esimerkiksi pistetilanteen.

Rivinsisäiset koodausehdotukset olivat käteviä, mutta en juurikaan käyttänyt niitä, sillä halusin oppia itse kirjoittamaan koodin syntaksista lähtien. Rivinsisäiset ehdotukset muuttuivat hyvin nopeaan tahtiin sen mukaan, mitä itse kirjoitin ja ne saattoivat jopa hieman häiritä omaa ajatteluani. Copilotin kanssa keskustelussa tuntui, että olen itse enemmän vastuussa keskustelusta. Myöhemmin, kun perusasiat ovat hallussa, nämä rivinsisäiset ehdotukset ovat varmasti hyödyllisiä ja suurena apuna, kun kaikkea ei tarvitse kirjoittaa itse. Tähän hetkeen oman oppimisen kannalta ajattelen kuitenkin, että ne eivät ole paras tapa auttaa oppimista.

Copilotin ja rivinsisäisten ehdotusten ideat olivat mielestäni hyvin samanlaiset. Copilot ei myöskään ehdottanut mitään virheellistä. Ainoa haittapuoli oli, että Copilot vastasi kysymyksiin hyvin pitkästi ja laajasti, kun joskus tarvitsisi vain yhden pienen tarkennuksen johonkin asiaan. Tätä voisi toki muokata agenttiin tai kehotteeseen, jotta vastaukset eivät olisi niin rönsyileviä. Tekoälypohjaisia ohjelmointityökaluja ja niiden eri vaiheita ajatellen, sijoittaisin GitHub Copilotin tekoälyä hyödyntäväksi kehitysympäristöksi (AI-powered IDEs). Mukautettu agentti ei tee kaikkea työtä ohjelmoijan puolesta, vaan sen määrittely vaikuttaa esimerkiksi käyttäjälle annettuihin ohjeiisin. Mielestäni agentin määrittely ei muuta Copilotin sijaintia kontrolli-automaatio-akselilla, sillä se vain muokkaa Copilotin vastauksia halutunlaisiksi eikä välttämättä vaikuta koneen ja ihmisen väliseen työnjakoon niinkään.